

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه اصفهان

دانشکده فنی مهندسی

گروه مهندسی برق

پروژه ی پایانی درس اصول میکرو کامپیوتر و میکروکنترلر

رأدار 180 درجه با سروموتور و التراسونیک

استاد:

دکتر محمد مطهری فر

دانشجو:

محمد جواد توکلی

حسین مرتضوی درچه

تیر ماه 1405

چکیده

این پروژه به پیاده سازی یک سیستم رادار ساده و کم هزینه با استفاده از میکروکنترلر ATmega32، سنسور فراصوت HC-SR04، نمایشگر OLED و موتور سروو می پردازد. سیستم با چرخش سروو در محدوده ۰ تا ۱۸۰ درجه، فاصله موانع را اندازه گیری کرده و به صورت گرافیکی روی (OLED) نمایش می دهد. کد به زبان C نوشته شده و از وقفه خارجی INT0 و وقفه سرریز Timer0 برای اندازه گیری زمان پالس اکو استفاده می کند. این طرح با هدف آموزش سیستم های نهفته، رباتیک و نمایش اصول عملکرد رادار فراصوت طراحی شده است.

کلیدواژه ها

رادار، سنسور فراصوت، HC-SR04، ATmega32، OLED، سروو موتور، اندازه گیری فاصله، سیستم های نهفته

فهرست مطالب

عنوان	صفحه
فصل اول.....	1
مقدمه و کلیات	1
1-1- هدف پروژه.....	1
2-1- کاربردها	1
3-1- نمای کلی سیستم.....	1
4-1- ساختار گزارش.....	1
فصل دوم	2
مشخصات سخت افزاری.....	2
1-2- میکروکنترلر و فرکانس.....	2
2-2- سنسورها و ماژول ها	2
3-2- پین های استفاده شده	3
4-2- منابع تغذیه	3
فصل سوم	3
جزئیات پیاده سازی.....	3
1-3- اندازه گیری فاصله با Timer0	3
1-1-3 فرمول های محاسباتی.....	4
1-2-3 تحلیل عملکرد	4
2-3- تولید PWM برای سروو موتور.....	4
1-2-3 فرمول های محاسباتی.....	5
2-2-3 تحلیل سیگنال.....	5
3-3- نمایش روی OLED	6
1-3-3 معماری نمایشگر	6
2-3-3 ساختار بافر	6
3-3-3 تبدیل مختصات قطبی به دکارتی :	6
فصل چهارم	7
آنالیز کد.....	7

7	1-4- ساختار کد.....
7	2-4- متغیرهای برنامه.....
8	3-4- توابع اصلی (Main Functions).....
8	1-3-4 تابع ext_int0_isr().....
8	2-3-4 تابع timer0_ovf_isr().....
8	3-3-4 تابع get_distance().....
9	4-3-4 تابع I2C_Init().....
9	5-3-4 تابع I2C_Start().....
9	6-3-4 تابع I2C_Stop().....
9	7-3-4 تابع I2C_Write().....
10	8-3-4 تابع OLED_Cmd().....
10	9-3-4 تابع OLED_Data().....
10	10-3-4 تابع OLED_Pixel().....
10	11-3-4 تابع OLED_Display().....
10	12-3-4 تابع OLED_Init().....
11	13-3-4 تابع DrawRadarPoint().....
11	14-3-4 تابع servo_init().....
11	15-3-4 تابع servo_write().....
11	16-3-4 تابع main().....
12	فصل پنجم.....
12	شبیه سازی کد.....
14	فصل ششم.....
14	ساخت عملی.....
14	1-6- تغییرات انجام شده در نرم افزار.....
15	1-1-6 توابع حذف شده.....
15	2-1-6 توابع اضافه شده.....
15	3-1-6 متغیرهای حذف شده.....
16	2-6- ساختار جدید سیستم.....
16	3-6- ارتباط سریال.....
16	1-3-6 تابع uart_init().....
17	2-3-6 تابع uart_send_char().....

17	uart_send_number() تابع 3-3-6
17	uart_send_string() تابع 4-3-6
17	توضیحات تکمیلی 4-6
18	تصاویر 5-6
19	References
20	پیوست
20	پیوست الف
22	پیوست ب
23	پیوست ج
25	پیوست د
28	پیوست و

فهرست شکل‌ها

صفحه	عنوان
12	شکل 1: شبیه سازی در پروتئوس.....
13	شکل 2: شبیه سازی در پروتئوس.....
16	شکل 3: ساختار کد عملی.....
18	شکل 4: نمایش گرافیکی با رایانه از شبیه ساز.....
18	شکل 5: شبیه سازی قبل از اجرای عملی

فهرست جدول‌ها

صفحه	عنوان
2	جدول 1: مشخصات میکروکنترلر
2	جدول 2: سنسور ها
3	جدول 3: اتصالات پروژه
4	جدول 4: عملکرد سنسور التراسونیک.....
5	جدول 5: سیگنال سرو موتور
7	جدول 6: متغیر ها
14	جدول 7: تفاوت دو مدل ATmega32, ATmega16

فصل اول

مقدمه و کلیات

1-1- هدف پروژه

طراحی و پیاده سازی یک رادار ساده با استفاده از میکروکنترلر ATmega32 که با چرخش سرووموتور، محیط را اسکن کرده و فاصله موانع را توسط سنسور التراسونیک اندازه گیری و روی نمایشگر OLED نمایش می دهد.

1-2- کاربردها

- سیستم های تشخیص مانع
- آموزش سیستم های Embedded
- پروژه های رباتیک

1-3- نمای کلی سیستم

سیستم شامل اجزای زیر است:

واحد پردازشگر: ATmega32

واحد اندازه گیری: سنسور HC-SR04

واحد نمایش: OLED SSD1306 (128×64)

واحد مکانیکی: سروو موتور

1-4- ساختار گزارش

در ادامه ابتدا به سخت افزار پروژه اشاره شده است و بعد از آن به جزئیات نکات قابل ملاحظه هنگام پیاده سازی اشاره شده است و در آخر عکس و مستندات از شبیه سازی و فیلم ساخت عملی پروژه موجود می باشد. باتوجه به اینکه برای ساخت عملی از ATmega16 استفاده شده است دو کد در گزارش بررسی می شود ابتدا کد کامل ATmega32 بررسی شده و در ادامه به کدی که در عمل استفاده شده است میپردازیم. توضیحات هر بخش از کد در پیوست ها موجود می باشد و کد اولیه که با میکرو ATmega32 برنامه ریزی شده است در فایل با نام [Radar32.zip](#) بدون کامنت می باشد. کد پروژه برای بخش ساخت عملی در فایل با نام [Radar16.zip](#) موجود می باشد.

فصل دوم

مشخصات سخت افزاری

1-2- میکروکنترلر و فرکانس

جدول 1: مشخصات میکروکنترلر [1]

مشخصه	مقدار
نوع	ATmega32
فرکانس	۸ مگاهرتز
حافظه برنامه	۳۲ کیلوبایت FLASH
حافظه داده	۲ کیلوبایت SRAM
حافظه EEPROM	۱ کیلوبایت

2-2- سنسورها و ماژول ها

جدول 2: سنسورها

قطعه	مشخصات
HC-SR04	ولتاژ ۵V، برد ۴۰-۲cm، زاویه ۱۵ درجه
OLED SSD1306	I2C، ۳.۳-۵V، ۶۴×۱۲۸
سروو موتور	۴.۸-۶V، ۵۰Hz، زاویه ۱۸۰ درجه

3-2- پین‌های استفاده‌شده

جدول 3: اتصالات پروژه

پایه	پورت	عملکرد	جهت
۱	PD1	TRIG (HC-SR04)	خروجی
۲	PD2	ECHO (HC-SR04)	ورودی (INT0)
۳	PD5	PWM سروو (OC1A)	خروجی
۴	PC1	SCL (I2C OLED)	دوطرفه
۵	PC0	SDA (I2C OLED)	دوطرفه

4-2- منابع تغذیه

میکروکنترلر: ۵ ولت

OLED: 3/3~5

سروو موتور: ۵-۶ ولت (منبع مجزا برای جلوگیری از نویز)

سنسور HC-SR04: ۵ ولت (از برد اصلی)

فصل سوم

جزئیات پیاده‌سازی

3-1- اندازه‌گیری فاصله با Timer0

در این پروژه، تشخیص لبه‌های سیگنال Echo توسط وقفه خارجی INT0 انجام می‌شود. هنگام مشاهده لبه بالارونده، Timer0 شروع به شمارش کرده و در لبه پایین‌رونده متوقف می‌شود. مقدار شمارنده و تعداد سرریزهای Timer0 با یکدیگر ترکیب شده و زمان رفت و برگشت موج اولتراسونیک محاسبه می‌شود. سپس این مقدار به فاصله بر حسب سانتی‌متر تبدیل می‌شود. [1] [2]

پری‌اسکالر: ۶۴ (هر تیک = ۸ میکروثانیه در فرکانس ۸ مگاهرتز)

محاسبه فاصله:

`pulse_width=((overflow<<8)|TCNT0);`

```
return pulse_width/58;
```

زمان انتظار: ۶۰ میلی ثانیه برای پاسخ سنسور

1-1-3 فرمول های محاسباتی

$$\text{Timer Tick Time} = \text{Prescaler} / F_{\text{CPU}}$$

$$= 64 / 8,000,000 \text{ Hz}$$

$$= 8 \mu\text{s}$$

$$\text{Pulse Width} (\mu\text{s}) = (\text{timer0_overflow} \times 256 + \text{TCNT0}) \times 8 \mu\text{s}$$

$$\text{Distance (cm)} = (\text{Pulse Width} \times \text{Speed of Sound}) / 2$$

$$= (\text{Pulse Width} \times 0.0344 \text{ cm}/\mu\text{s}) / 2$$

$$= \text{Pulse Width} / 58.14$$

$$\approx \text{Pulse Width} / 58$$

کد این بخش همراه با کامنت در پیوست الف قابل مشاهده می باشد. [2]

2-1-3 تحلیل عملکرد

جدول 4: عملکرد سنسور التراسونیک

پارامتر (Parameter)	مقدار (Value)	توضیح (Explanation)
دقت اندازه گیری (Measurement Accuracy)	± 2 سانتی متر ($\pm 2\text{cm}$)	وابسته به کریستال و نویز (Depends on crystal and noise)
حداکثر برد (Maximum Range)	~ 400 سانتی متر ($\sim 400\text{cm}$)	محدودیت سنسور (HC-SR04 limit)
حداقل برد (Minimum Range)	~ 2 سانتی متر ($\sim 2\text{cm}$)	محدودیت سنسور (Sensor limit)
زمان اندازه گیری (Measurement Time)	۶۰ ms	شامل زمان انتظار (Including wait time)

2-3- تولید PWM برای سروو موتور

برای چرخاندن سنسور در محدوده ۱۸۰ درجه از یک سروو موتور استاندارد استفاده شده است. تولید سیگنال PWM توسط Timer1 در مد Fast PWM انجام می شود. فرکانس PWM برابر ۵۰ هرتز است که برای اکثر سروو موتورها مناسب است. عرض پالس خروجی بین ۱۰۰۰ تا ۲۰۰۰ میکروثانیه

تغییر می‌کند و متناسب با زاویه موردنظر مقداردهی می‌شود. زاویه سرووموتور در حلقه اصلی برنامه از صفر تا ۱۸۰ درجه افزایش یافته و سپس دوباره تا صفر کاهش می‌یابد تا عملیات اسکن رفت و برگشتی انجام شود. [3]

1-2-3 فرمول‌های محاسباتی

$$\text{Timer Frequency} = F_{\text{CPU}} / \text{Prescaler}$$

$$= 8\text{MHz} / 8$$

$$= 1\text{MHz}$$

$$\text{Timer Period} = 1 / \text{Timer Frequency}$$

$$= 1\mu\text{s}$$

$$\text{PWM Period} = \text{ICR1} \times \text{Timer Period}$$

$$= 20000 \times 1\mu\text{s}$$

$$= 20\text{ms}$$

$$\text{Pulse Width} = \text{OCR1A} \times \text{Timer Period}$$

$$= \text{OCR1A} \times 1\mu\text{s}$$

Angle to Pulse Conversion:

$$\text{Pulse Width} (\mu\text{s}) = 1000 + (\text{Angle} \times 1000) / 180$$

$$\text{OCR1A} = \text{Pulse Width} (\text{since each tick} = 1\mu\text{s})$$

کد این بخش همراه با کامنت در پیوست ب قابل مشاهده می باشد. [3]

2-2-3 تحلیل سیگنال

جدول 5: سیگنال سرو موتور

زاویه (Angle)	عرض پالس (Pulse Width)	OCR1A
۰°	۱۰۰۰ μs	۱۰۰۰
۴۵°	۱۲۵۰ μs	۱۲۵۰
۹۰°	۱۵۰۰ μs	۱۵۰۰

۱۷۵۰	$\mu s 1750$	۱۳۵°
۲۰۰۰	$\mu s 2000$	۱۸۰°

3-3- نمایش روی OLED

1-3-3 معماری نمایشگر

نمایشگر SSD1306 از طریق پروتکل I2C به میکروکنترلر متصل شده است. ابتدا واحد TWI مقداردهی اولیه می شود و سپس دستورات راه اندازی نمایشگر ارسال می شوند. اطلاعات تصویری ابتدا داخل یک بافر ۱۰۲۴ بایتی ذخیره شده و در پایان هر مرحله اسکن به صورت یکجا روی نمایشگر ارسال می شوند. این روش باعث کاهش تعداد تبادلات روی باس I2C و افزایش سرعت نمایش می شود. و با محدودیت فاصله زیر در کد و نمایش روبرو هستیم. [5] [4]

max_range = 100cm;

بنابراین اگر فاصله ای بیشتر از ۱۰۰ سانتی متر باشد، نقطه خارج از شعاع نمایش قرار می گیرد و روی OLED به درستی نمایش داده نمی شود.

2-3-3 ساختار بافر

$$\text{آدرس بافر} = x + (y/8) \times 128$$

بایت = ۸ پیکسل عمودی (بیت ۰ = بالاترین پیکسل)

مثال:

Pixel(10, 25)

$$\text{- page} = 25/8 = 3$$

$$\text{- bit} = 25 \% 8 = 1$$

$$\text{- buffer}[10 + 3 \times 128] |= (1 << 1)$$

3-3-3 تبدیل مختصات قطبی به دکارتی :

Given: (r, θ) where:

- r = distance in cm

- θ = angle in degrees (0° to 180°)

Cartesian coordinates:

$$X = \text{Center_X} + r \times \text{Scale} \times \cos(\theta)$$

$$Y = \text{Center_Y} - r \times \text{Scale} \times \sin(\theta)$$

Where:

- Center_X = 64 (midpoint of 128 pixels)
- Center_Y = 63 (midpoint of 64 pixels, with Y inverted)
- Scale = 60/100 = 0.6 (60 pixels for 100cm range)

کد این بخش همراه با کامنت در پیوست ج قابل مشاهده می باشد. [4] [5]

فصل چهارم

آنالیز کد

1-4- ساختار کد

2-4- متغیرهای برنامه

جدول 6: متغیرها

متغیر (Variable)	نوع (Type)	کاربرد (Application)
pulse_width	unsigned long	ذخیره عرض پالس Echo دریافتی از سنسور HC-SR04 و محاسبه فاصله
edge_flag	unsigned char	تشخیص وضعیت لبه سیگنال Echo (لبه بالارونده یا پایین‌رونده)
measurement_done	bit	مشخص می‌کند عملیات اندازه‌گیری فاصله به پایان رسیده است یا خیر
timer0_overflow	unsigned int	شمارش تعداد سرریزهای Timer0 برای اندازه‌گیری پالس‌های طولانی
buffer[1024]	unsigned char	بافر گرافیکی نمایشگر OLED که داده‌های تصویری قبل از ارسال در آن ذخیره می‌شوند
distance	unsigned int	ذخیره فاصله اندازه‌گیری شده (در تابع main)
ang	int	زاویه فعلی سرووموتور در هنگام اسکن
direction	int	تعیین جهت حرکت سرووموتور (افزایش یا کاهش زاویه)

3-4- توابع اصلی (Main Functions)

1-3-4 تابع ext_int0_isr()

- هدف: اندازه‌گیری عرض پالس Echo با استفاده از وقفه خارجی. INT0.
- ورودی: ندارد
- خروجی: ندارد
- عملکرد: در اولین وقفه (لبه بالارونده)، Timer0 صفر شده و شروع به شمارش می‌کند. نوع وقفه به لبه پایین‌رونده تغییر می‌کند. در دومین وقفه، Timer0 متوقف شده و مقدار زمان پالس در متغیر pulse_width ذخیره می‌شود. سپس پرچم measurement_done فعال می‌شود تا برنامه اصلی متوجه پایان اندازه‌گیری گردد.

2-3-4 تابع timer0_ovf_isr()

- هدف: ثبت سرریزهای Timer0.
- ورودی: ندارد
- خروجی: ندارد
- عملکرد: هر بار که Timer0 از 255 عبور می‌کند، این وقفه اجرا شده و متغیر¹ یک واحد افزایش می‌یابد.

3-3-4 تابع get_distance()

- هدف: اندازه‌گیری فاصله جسم.
 - ورودی: ندارد
 - خروجی: فاصله بر حسب سانتی‌متر (unsigned int)
 - عملکرد: متغیرهای اندازه‌گیری ریست می‌شوند.
- پالس Trigger برای سنسور ارسال می‌شود.
- برنامه منتظر پایان اندازه‌گیری می‌ماند.
- عرض پالس محاسبه می‌شود.

فاصله با رابطه تقریبی $pulse_width / 58$ محاسبه می شود .

در صورت بروز خطا مقدار صفر بازگردانده می شود .

4-3-4 تابع I2C_Init()

- هدف: راه اندازی رابط I2C.
- ورودی: ندارد
- خروجی: ندارد
- عملکرد: رجیسترهای TWI مقداردهی شده و سرعت ارتباط روی حدود 400kHz تنظیم می شود.

5-3-4 تابع I2C_Start()

- هدف: ارسال شرط Start روی باس I2C.
- ورودی: ندارد
- خروجی: ندارد
- عملکرد: شروع ارتباط بین میکروکنترلر و OLED را برقرار می کند.

6-3-4 تابع I2C_Stop()

- هدف: اتمام ارتباط I2C.
- ورودی: ندارد
- خروجی: ندارد
- عملکرد: پس از پایان ارسال اطلاعات، شرط Stop را روی باس ایجاد می کند.

7-3-4 تابع I2C_Write()

- هدف: ارسال یک بایت داده روی I2C.
- ورودی: یک بایت داده (unsigned char)
- خروجی: ندارد
- عملکرد: داده را داخل رجیستر TWDR قرار داده و تا پایان ارسال منتظر می ماند.

8-3-4 تابع OLED_Cmd()

- هدف: ارسال فرمان به نمایشگر OLED.
- ورودی: کد فرمان
- خروجی: ندارد
- عملکرد: فرمان‌هایی مانند روشن شدن نمایشگر، تنظیم صفحه و آدرس‌دهی را ارسال می‌کند.

9-3-4 تابع OLED_Data()

- هدف: ارسال داده تصویری به OLED.
- ورودی: یک بایت داده
- خروجی: ندارد
- عملکرد: داده‌های گرافیکی را برای نمایش ارسال می‌کند.

10-3-4 تابع OLED_Pixel()

- هدف: روشن کردن یک پیکسل در بافر تصویر.
- ورودی: مختصات X و Y
- خروجی: ندارد
- عملکرد: پیکسل موردنظر را در آرایه buffer فعال می‌کند.

11-3-4 تابع OLED_Display()

- هدف: ارسال کل بافر به نمایشگر.
- ورودی: ندارد
- خروجی: ندارد
- عملکرد: تمام 1024 بایت بافر را به OLED منتقل می‌کند.

12-3-4 تابع OLED_Init()

- هدف: راه‌اندازی اولیه نمایشگر.
- ورودی: ندارد

- خروجی: ندارد
- عملکرد: تمام تنظیمات اولیه SSD1306 مانند کنتراست، حافظه، جهت نمایش و فعال شدن Charge Pump را انجام می دهد.

13-3-4 تابع DrawRadarPoint()

- هدف: تبدیل فاصله و زاویه به مختصات نمایشگر.
- ورودی: فاصله و زاویه
- خروجی: ندارد
- عملکرد: با استفاده از روابط مثلثاتی $\sin$ و $\cos$ مختصات نقطه روی OLED محاسبه شده و پیکسل متناظر روشن می شود.

14-3-4 تابع servo_init()

- هدف: راه اندازی سرووموتور.
- ورودی: ندارد
- خروجی: ندارد
- عملکرد: Timer1 را در مد Fast PWM تنظیم کرده و دوره تناوب 20 میلی ثانیه (50Hz) را ایجاد می کند. سپس سروو را در زاویه میانی (۹۰ درجه) قرار می دهد.

15-3-4 تابع servo_write()

- هدف: تغییر زاویه سرووموتور.
- ورودی: زاویه بین 0 تا 180 درجه
- خروجی: ندارد
- عملکرد: زاویه را به عرض پالس PWM بین 1000 تا 2000 میکروثانیه تبدیل کرده و مقدار OCR1A را به روزرسانی می کند.

16-3-4 تابع main()

- هدف: کنترل کل سیستم.
- عملکرد: مقداردهی اولیه پورت ها، تایمرها، وقفه ها، I2C، OLED و سرووموتور. فعال سازی وقفه های سراسری.

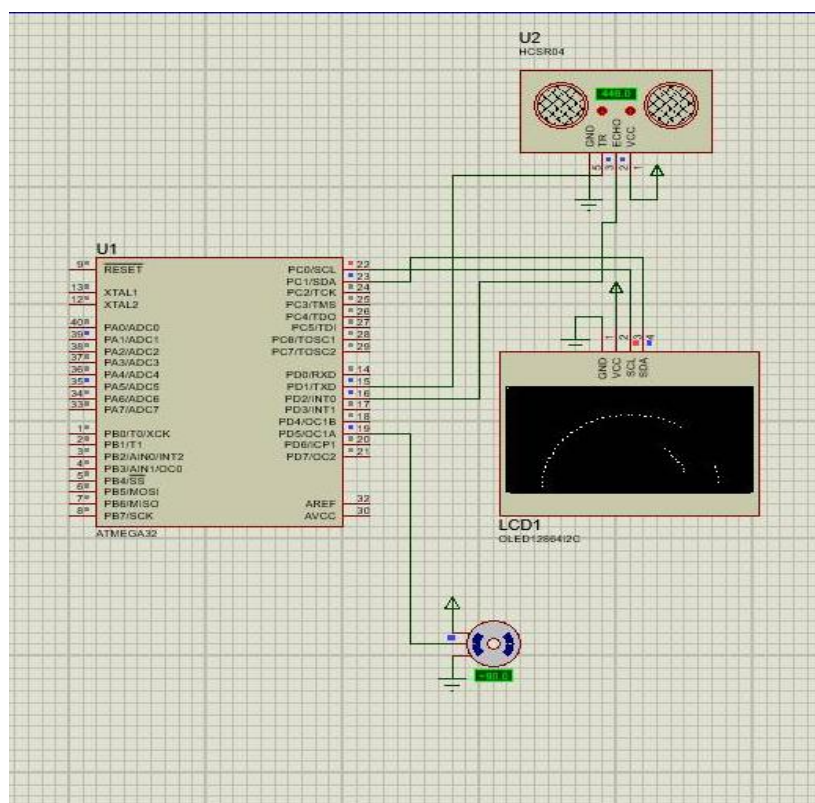
- اجرای حلقه اصلی شامل :

1. تغییر زاویه سرووموتور و تعیین جهت حرکت
2. اندازه گیری فاصله
3. تبدیل فاصله و زاویه به مختصات
4. رسم نقطه روی بافر OLED
5. ارسال بافر به نمایشگر
6. تکرار عملیات اسکن

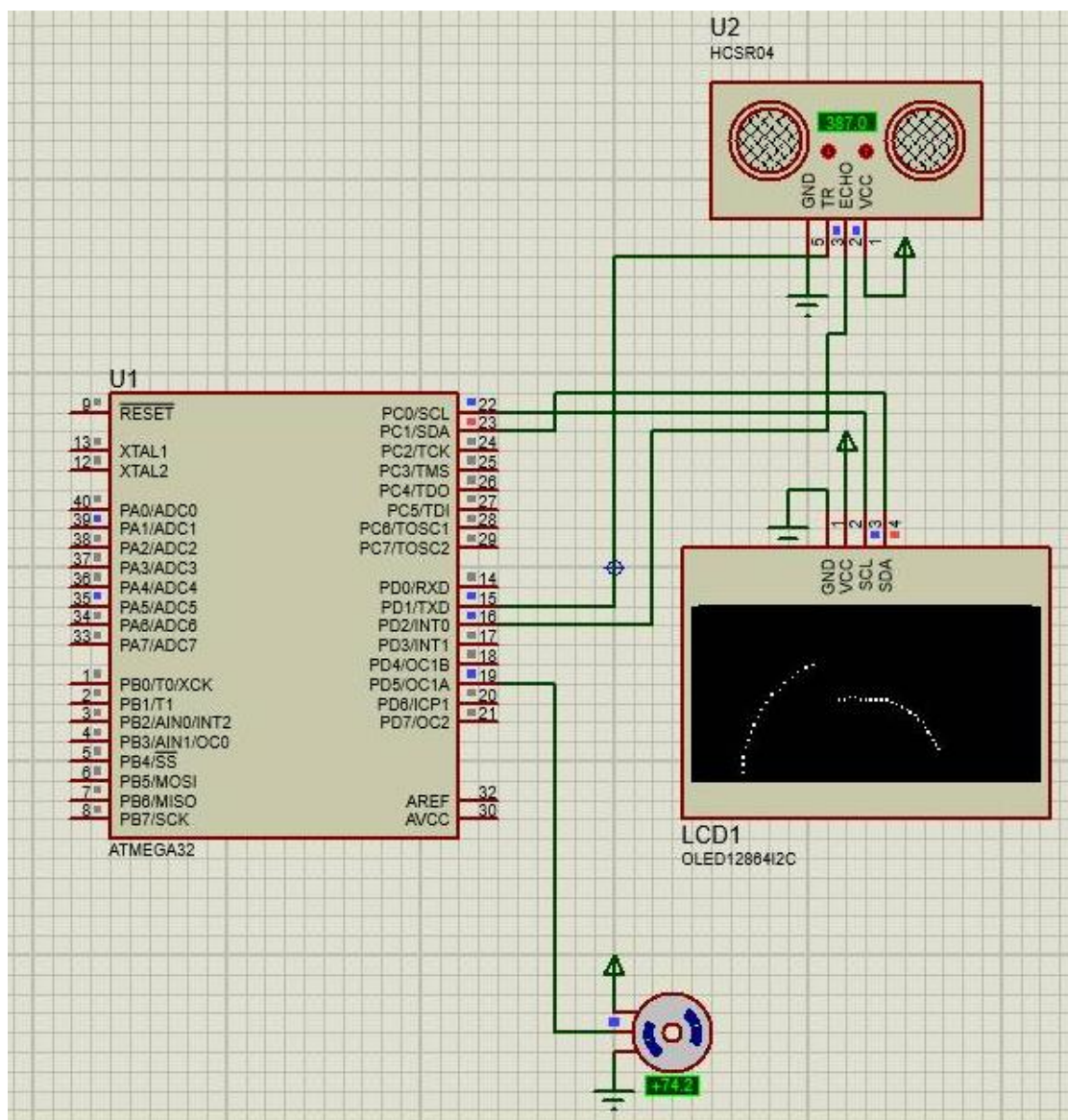
فصل پنجم

شبیه سازی کد

در تصاویر زیر شبیه سازی شده پروژه را با کد ویژن قابل مشاهده می باشد .



شکل 1: شبیه سازی در پروتئوس



شکل 2: شبیه سازی در پروتئوس

فصل ششم

ساخت عملی

پس از تکمیل شبیه‌سازی پروژه با میکروکنترلر ATmega32، در مرحله ساخت عملی به دلیل در دسترس بودن قطعات، از میکروکنترلر ATmega16 استفاده شد. اگرچه ساختار داخلی این دو میکروکنترلر بسیار مشابه است، اما حافظه Flash و SRAM در ATmega16 کمتر از ATmega32 است. به همین دلیل امکان استفاده هم‌زمان از تمامی بخش‌های برنامه اولیه، به خصوص کتابخانه و بافر نمایشگر OLED، وجود نداشت. بنابراین جهت کاهش حجم برنامه، بخش نمایشگر OLED حذف و نمایش اطلاعات رادار به کمک ارتباط سریال (UART) و نرم‌افزار Processing روی رایانه انجام شد.

کد مربوط به نرم‌افزار processing در پیوست د قابل مشاهده می باشد. [6]

محدودیت حافظه Flash و SRAM باعث شد بافر 1024 بیتی حذف شوند. این تغییر علاوه بر کاهش حجم برنامه، قابلیت نمایش گرافیکی توسط رایانه و ارتباط سریال UART را فراهم کرد [3].

جدول 7: تفاوت دو مدل ATmega16، ATmega32 [1] [3]

ویژگی	ATmega32	ATmega16
Flash	32 KB	16 KB
SRAM	2 KB	1 KB
EEPROM	1 KB	512 B

6-1- تغییرات انجام شده در نرم‌افزار

- حذف کامل کتابخانه OLED
- حذف توابع I2C
- حذف بافر تصویر
- حذف توابع رسم نقطه
- اضافه شدن UART
- ارسال زاویه و فاصله از طریق سریال
- نمایش اطلاعات در Processing

1-1-6 توابع حذف شده

در نسخه عملی، به دلیل حذف نمایشگر OLED و استفاده از نرم افزار Processing، توابع زیر از برنامه حذف شدند:

- I2C_Init()
- OLED_Cmd()
- OLED_Data()
- OLED_Pixel()
- OLED_Init()
- OLED_Clear()
- DrawRadarPoint()

2-1-6 توابع اضافه شده

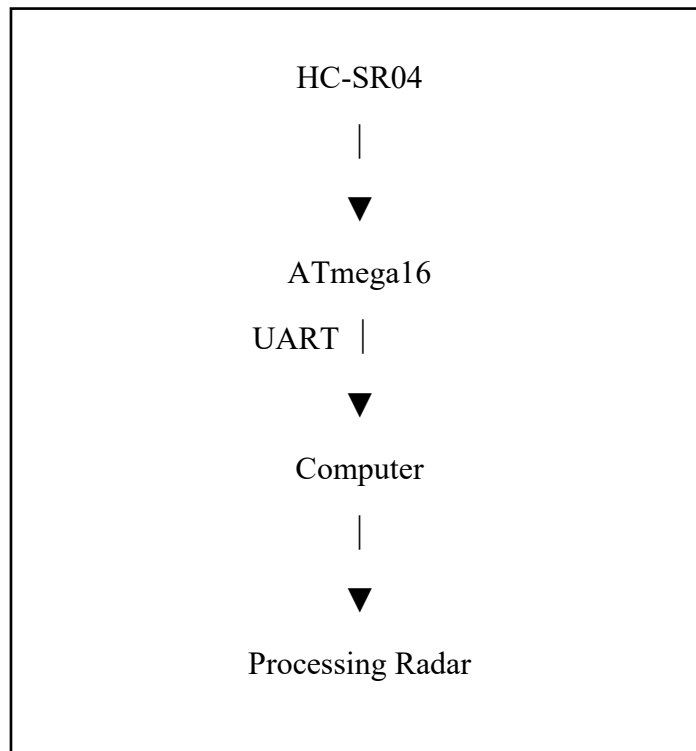
در نسخه عملی، به منظور برقراری ارتباط سریال با رایانه و نمایش اطلاعات در نرم افزار Processing، توابع زیر به برنامه اضافه شدند:

- uart_init()
- uart_send_char()
- uart_send_number()
- uart_send_string()

3-1-6 متغیرهای حذف شده

با حذف نمایشگر OLED، متغیرها و ساختارهای مربوط به کنترل و رسم تصویر نیز حذف شدند (در صورت وجود در نسخه اولیه)، مانند:

- بافر تصویر (Frame Buffer)
 - متغیرهای مربوط به مختصات و رسم روی OLED
- بخش های اضافه شده به کد در پیوست و قابل مشاهده می باشد.



شکل 3: ساختار کد عملی

6-3- ارتباط سریال

در نسخه عملی، پس از اندازه‌گیری فاصله، میکروکنترلر زاویه سروموتور و فاصله اندازه‌گیری شده را از طریق UART با نرخ 9600 بیت بر ثانیه به رایانه ارسال می‌کند. نرم‌افزار Processing این داده‌ها را دریافت کرده و موقعیت مانع را به صورت گرافیکی نمایش می‌دهد. در ادامه توابع اضافه شده توضیح داده شده است.

6-3-1 تابع `uart_init()`

- هدف: راه‌اندازی واحد USART برای برقراری ارتباط سریال با رایانه.
- عملکرد: این تابع رابط USART میکروکنترلر را در حالت Asynchronous با نرخ انتقال 9600 بیت بر ثانیه، 8 بیت داده، بدون بیت توازن (Parity) و یک بیت توقف (Stop Bit) تنظیم می‌کند تا داده‌های اندازه‌گیری شده برای نرم‌افزار Processing ارسال شوند.

تابع `uart_send_char()` 2-3-6

- هدف: ارسال یک کاراکتر از طریق رابط سریال.
- عملکرد: تابع ابتدا منتظر خالی شدن ثبات ارسال (UDR) می ماند و سپس کاراکتر موردنظر را ارسال می کند. این تابع پایه ای ترین تابع ارسال اطلاعات از طریق UART است.

تابع `uart_send_number()` 3-3-6

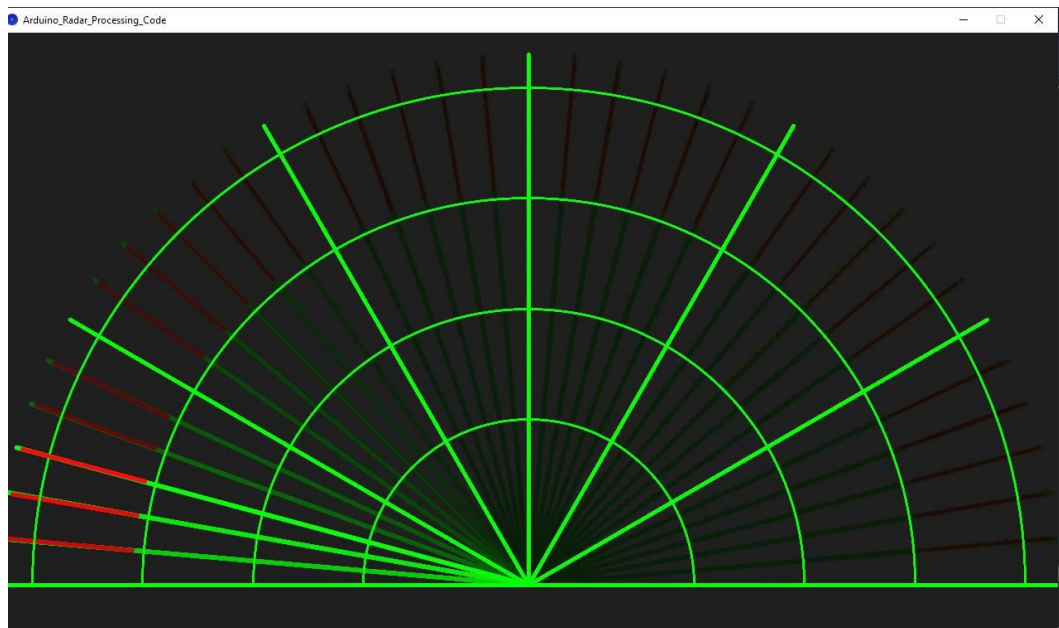
- هدف: ارسال یک عدد صحیح از طریق UART.
- عملکرد: از آنجا که رابط سریال فقط کاراکتر ارسال می کند، این تابع عدد ورودی را به ارقام ده دهی تبدیل کرده و هر رقم را به صورت یک کاراکتر ASCII ارسال می کند. استفاده از این روش باعث کاهش حجم برنامه شده و نیاز به توابع کتابخانه ای مانند `sprintf()` را از بین می برد.

تابع `uart_send_string()` 4-3-6

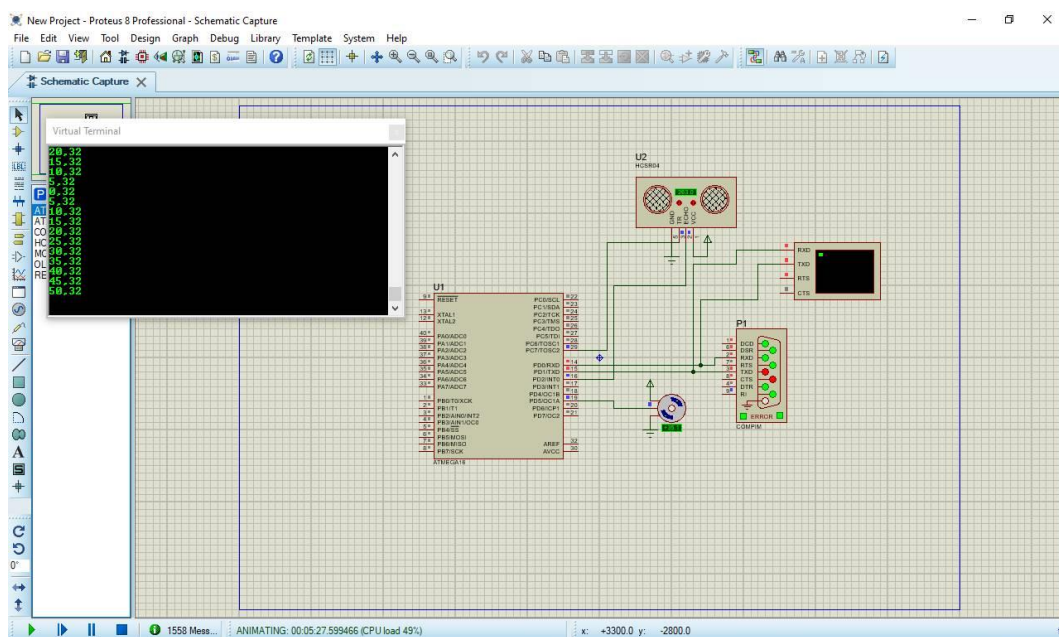
- هدف: ارسال یک رشته متنی از طریق رابط سریال.
- عملکرد: این تابع تمام کاراکترهای یک رشته را به ترتیب از طریق UART ارسال می کند و تا رسیدن به کاراکتر پایان رشته ('0') ادامه می یابد. از این تابع برای ارسال پیام های متنی و همچنین پایان هر بسته داده استفاده می شود.

4-6- توضیحات تکمیلی

پیش از پیاده سازی عملی، عملکرد سیستم در محیط Proteus مورد ارزیابی قرار گرفت. در این شبیه سازی، میکروکنترلر ATmega16A، سنسور HC-SR04، سرووموتور و ترمینال مجازی (Virtual Terminal) به کار گرفته شدند تا صحت عملکرد برنامه بررسی شود. داده های اندازه گیری شده شامل زاویه سرووموتور و فاصله جسم به صورت رشته ای با قالب Angle, Distance از طریق رابط سریال ارسال شده و در ترمینال مجازی نمایش داده شدند. نتایج شبیه سازی نشان داد که فرآیند اندازه گیری فاصله، حرکت رفت و برگشتی سرووموتور و ارسال اطلاعات از طریق UART به درستی انجام می شود.



شکل 4: نمایش گرافیکی با رایانه از شبیه ساز



شکل 5 : شبیه سازی قبل از اجرای عملی

در فایل زیپ یک [فیلیم](#) کوتاه از عملکرد پروژه بعد از ساخت تهیه شده و قابل مشاهده می باشد.

References

- [1] Microchip Technology Inc, "ATmega32Datasheet," Chandler, AZ, USA, 2006.
- [2] Microchip Technology Inc, "AVR130: Setup and Use of AVR Timers," Chandler, AZ, USA, 2016.
- [3] Microchip Technology Inc, "ATmega16(L) Datasheet," Microchip Technology Inc, Chandler, AZ, USA, 2006.
- [4] P. Fleury, "AVR Software Library," GitHub, 2020. [Online]. Available: <https://github.com/efthymios-ks/AVR-TWI>. [Accessed 2026].
- [5] A. Dynda, "SSD1306 OLED Display Library," GitHub, 2024. [Online]. Available: <https://github.com/lexus2k/ssd1306>. [Accessed 2026].
- [6] elktros, "Radar Visualization using Processing," 2020. [Online]. Available: <https://gist.github.com/elktros/5f56ede8e7266ca793a2cfd6f70c4319>.

پیوست

پیوست الف

```
//===== Distance Measurement =====

// وقفه خارجی INT0 برای تشخیص لبه‌های سیگنال Echo
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    if(edge_flag == 0)
    {
        // شروع اندازه‌گیری در لبه بالارونده
        TCNT0 = 0;
        timer0_overflow = 0;

        // شروع Timer0 با پری‌اسکیلر 64
        TCCR0 = (1<<CS01) | (1<<CS00);

        // تغییر وقفه به لبه پایین‌رونده
        MCUCR = (1<<ISC01);

        edge_flag = 1;
    }
    else
    {
        // توقف Timer0
        TCCR0 = 0;

        // محاسبه عرض پالس
        pulse_width = ((unsigned long)timer0_overflow<<8) | TCNT0;

        // آماده شدن برای اندازه‌گیری بعدی
        MCUCR = (1<<ISC01)|(1<<ISC00);
        edge_flag = 0;
        measurement_done = 1;
    }
}

// وقفه سرریز Timer0
interrupt [TIM0_OVF] void timer0_ovf_isr(void)
```

```

{
    timer0_overflow++;
}

// تابع اندازه‌گیری فاصله
unsigned int get_distance(void)
{
    pulse_width = 0;
    measurement_done = 0;
    edge_flag = 0;
    timer0_overflow = 0;

    // ارسال پالس Trigger
    TRIG = 0;
    delay_us(3);

    TRIG = 1;
    delay_us(12);

    TRIG = 0;

    // انتظار برای دریافت Echo
    delay_ms(60);

    if(measurement_done)
    {
        pulse_width=((unsigned long)timer0_overflow<<8)|TCNT0;

        if(pulse_width<50000)
            return pulse_width/58;
    }

    return 0;}

```

پیوست ب

```
//===== Servo Control =====

// راه اندازی برای تولید PWM
void servo_init(void)
{
    // پایه OC1A خروجی
    DDRD |= (1<<5);

    // Fast PWM در حالت Timer1
    TCCR1A = (1<<COM1A1)|(1<<WGM11);
    TCCR1B = (1<<WGM13)|(1<<WGM12)|(1<<CS11);

    // دوره تناوب 20ms
    ICR1H = (20000>>8);
    ICR1L = (unsigned char)20000;

    // زاویه اولیه 90 درجه
    OCR1A = 1500;
}

// تغییر زاویه سرو
void servo_write(unsigned char angle)
{
    unsigned int pulse;

    // محدود کردن زاویه
    if(angle>180)
        angle=180;

    // تبدیل زاویه به عرض پالس
    pulse = 1000 + ((unsigned long)angle*1000)/180;

    // اعمال مقدار PWM
    OCR1A = pulse;
}
```

پیوست ج

```
//===== OLED Display =====
```

```
// OLED فرمان به
```

```
void OLED_Cmd(unsigned char cmd)
```

```
{
```

```
    I2C_Start();
```

```
    I2C_Write(SSD1306_ADDR<<1);
```

```
    I2C_Write(0x00);
```

```
    I2C_Write(cmd);
```

```
    I2C_Stop();
```

```
}
```

```
// OLED داده به
```

```
void OLED_Data(unsigned char data)
```

```
{
```

```
    I2C_Start();
```

```
    I2C_Write(SSD1306_ADDR<<1);
```

```
    I2C_Write(0x40);
```

```
    I2C_Write(data);
```

```
    I2C_Stop();
```

```
}
```

```
// روشن کردن یک پیکسل در بافر
```

```
void OLED_Pixel(unsigned char x,unsigned char y)
```

```
{
```

```
    if(x>=128 || y>=64)
```

```
        return;
```

```
    buffer[x+(y/8)*128] |= (1<<(y%8));
```

```
}
```

```
// OLED بافر تصویر به
```

```
void OLED_Display(void)
```

```
{
```

```

    unsigned int i;
    unsigned char j;

    for(i=0;i<8;i++)
    {
        OLED_Cmd(0xB0+i);
        OLED_Cmd(0x00);
        OLED_Cmd(0x10);

        for(j=0;j<128;j++)
        {
            OLED_Data(buffer[j+i*128]);
        }
    }
}

// رسم نقطه رادار
void DrawRadarPoint(int distance,int angle)
{
    float max_range = 100.0;
    float scale = 60.0/max_range;
    float rad = angle*3.14159/180.0;

    int x = 64 + (int)(distance*scale*cos(rad));
    int y = 63 - (int)(distance*scale*sin(rad));

    OLED_Pixel(x,y);
}

```

پیوست د

```
import processing.serial.*;
import processing.opengl.*;

Serial port;
String serialAngle;
String serialDistance;
String serialData;
float objectDistance;
int radarAngle, radarDistance;
int index=0;

void setup()
{
    size (1280, 720);
    smooth();

    String portName = "COM5";

    port = new Serial(this, portName, 9600);

    port.bufferUntil('\n'); // خواندن داده تا رسیدن به خط جدید
}

void draw()
{
    noStroke();

    fill(0,4);
    rect(0, 0, 1280, 720);
    fill(10,255,10);

    // رادار: کمان‌ها و خطوط
    pushMatrix();

    translate(640,666); // مرکز رادار
    noFill();
    strokeWeight(2);
```



```

stroke(10,255,10); // رنگ سبز

// کمان‌های رادار (فاصله‌ها)
arc(0,0,1200,1200,PI,TWO_PI); // بزرگترین دایره
arc(0,0,934,934,PI,TWO_PI); // دایره دوم
arc(0,0,666,666,PI,TWO_PI); // دایره سوم
arc(0,0,400,400,PI,TWO_PI); // کوچکترین دایره

strokeWeight(4);
// خطوط زاویه‌ای رادار
line(-640,0,640,0); // خط افقی
line(0,0,-554,-320); // 30 درجه
line(0,0,-320,-554); // 60 درجه
line(0,0,0,-640); // 90 درجه (عمودی)
line(0,0,320,-554); // 120 درجه
line(0,0,554,-320); // 150 درجه

popMatrix();

// خط اسکنر رادار (خط متحرک)
pushMatrix();

strokeWeight(5);
stroke(10,255,10); // رنگ سبز
translate(640,666);
line(0,0,640*cos(radians(radarAngle)),-640*sin(radians(radarAngle)));

popMatrix();

// خط تشخیص مانع (قرمز)
pushMatrix();

translate(640,666);
strokeWeight(5);
stroke(255,10,10); // رنگ قرمز
objectDistance = radarDistance*15; // تبدیل فاصله به پیکسل

if(radarDistance<40)
{
    // رسم خط از مانع تا لبه رادار

```

```

        line(objectDistance*cos(radians(radarAngle)), -objectDistance*sin(radians(radarAngle)), 633*cos(radians(radarAngle)), -633*sin(radians(radarAngle)));
    }

    popMatrix();
}

// تابع دریافت داده از پورت سریال
void serialEvent (Serial port)
{
    serialData = port.readStringUntil('\n'); // خواندن تا خط جدید

    if(serialData == null) return; // اگر داده وجود نداشت برگرد

    serialData = trim(serialData); // حذف فاصله های اضافی

    if(serialData.length()==0) return; // اگر رشته خالی بود برگرد

    index = serialData.indexOf(","); // پیدا کردن ویرگول جداکننده

    if(index== -1) return;

    // جداسازی زاویه و فاصله
    serialAngle = serialData.substring(0, index);
    serialDistance = serialData.substring(index + 1);

    // تبدیل به عدد صحیح
    radarAngle = int(serialAngle);
    radarDistance = int(serialDistance);
}

```

پیوست و

```
// جهت حرکت سروو
int direction = 0;

// تنظیم زاویه سروو
void servo_write(unsigned char angle)
{
    if(angle > 180) angle = 180;
    angle = 180 - angle;
    OCR1A = 1000 + ((unsigned long)angle * 1000) / 180;
}

// راه اندازی سریال
void uart_init(void)
{
    UCSRB = (1<<RXEN) | (1<<TXEN);
    UCSRC = (1<<URSEL) | (1<<UCSZ1) | (1<<UCSZ0);
    UBRRL = 51;
    UBRRH = 0;
}

// ارسال کاراکتر
void uart_send_char(char c)
{
    while (!(UCSRA & (1<<UDRE)));
    UDR = c;
}

// ارسال عدد
void uart_send_number(unsigned int num)
{
    char buf[8];
    unsigned char i = 0;

    if (num == 0)
    {
        uart_send_char('0');
        return;
    }
}
```

```

while (num > 0)
{
    buf[i++] = (num % 10) + '0';
    num /= 10;
}

while (i > 0)
{
    uart_send_char(buf[--i]);
}
}

// ارسال رشته
void uart_send_string(char *str)
{
    while (*str)
    {
        uart_send_char(*str++);
    }
}

uart_init();           // شروع سریال
#asm("sei")           // فعال سازی وقفه
uart_send_string("Radar Started\r\n");

while (1)
{
    // تغییر زاویه
    if(direction == 0)
        ang += 5;
    else
        ang -= 5;
    if(ang >= 180) direction = 1;
    if(ang <= 0) direction = 0;
    servo_write(ang);           // اعمال به سروو
    distance = get_distance();   // خواندن فاصله
    // ارسال داده
    uart_send_number(ang);
    uart_send_char(',');
}

```

```
    uart_send_number(distance);  
    uart_send_string("\r\n");  
    delay_ms(50);  
}
```